5. SYSTEM DESIGN & ARCHITECTURE

5.1 Architecture Diagram

This diagram illustrates the high-level MERN stack architecture, showing the distinct Client, Server (Node.js/Express with Socket.io), and Database (MongoDB) layers. It highlights how role-based frontend applications interact with backend services and external APIs.

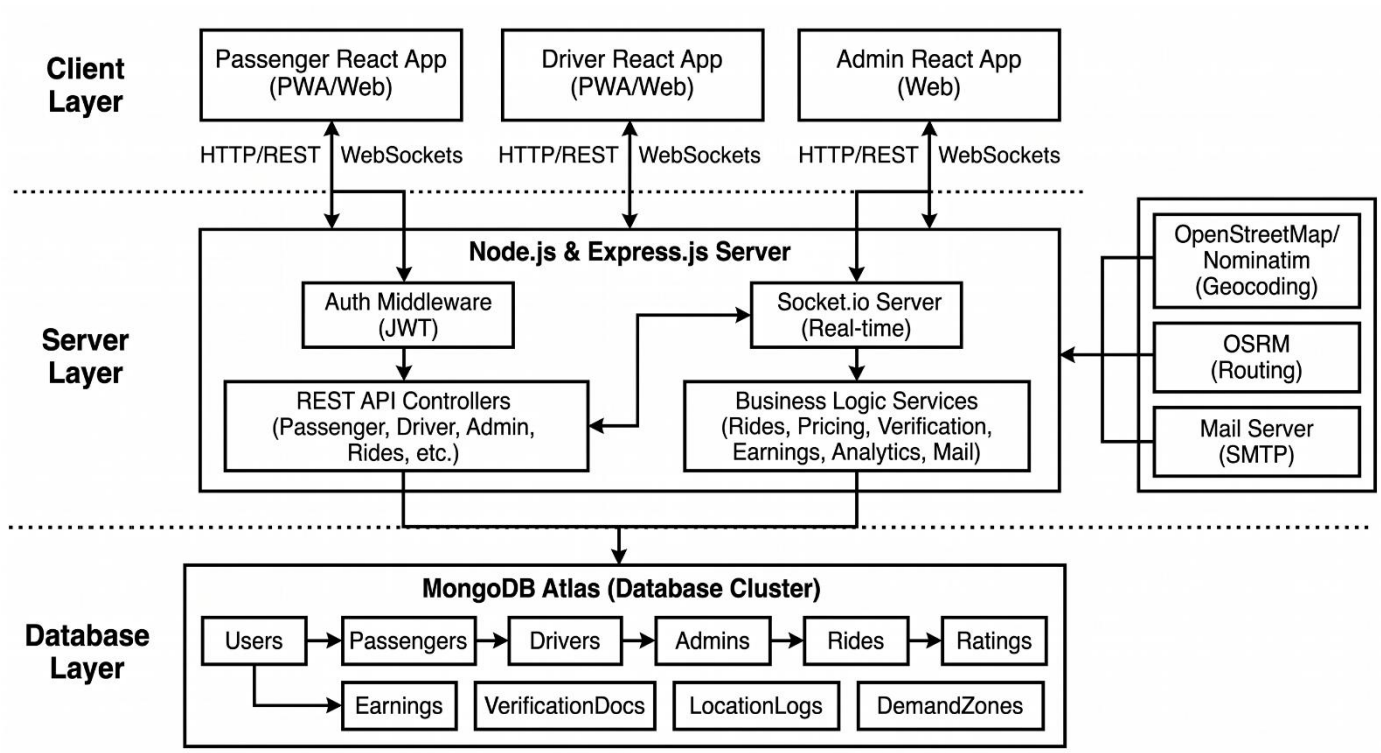

FIG 5.1 – Architecture Diagram

5.2 Data Flow Diagram (DFD)

The DFD maps the flow of information between Passengers, Drivers, Admins, and the core system. It visualizes how inputs like ride requests and outputs like status updates move through the processing engine and into data stores.

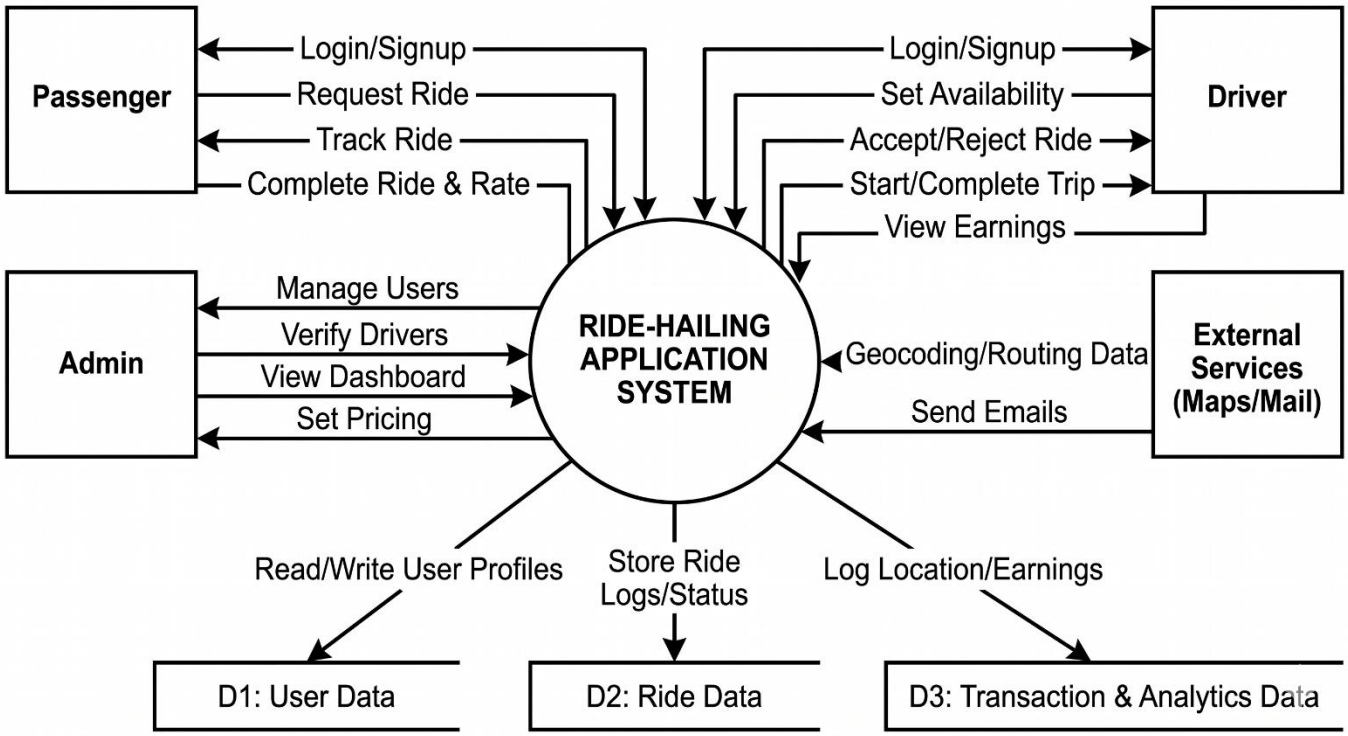

FIG 5.2 – Data Flow Diagram

5.3 UML Use Case Diagram

This diagram defines the system's functional boundaries by mapping actors (Passenger, Driver, Admin) to their specific actions. It covers core interactions such as booking rides, managing dynamic surge pricing, and viewing earnings.

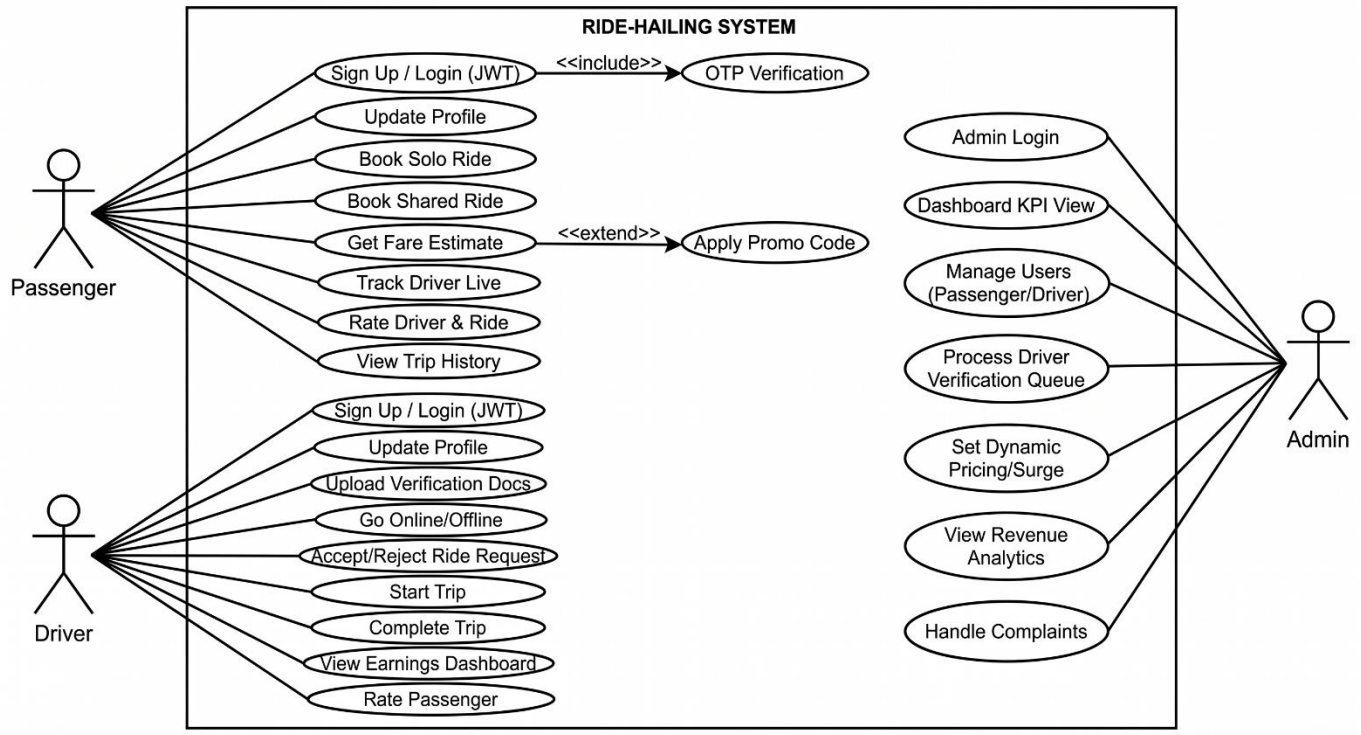

FIG 5.3 – UML Diagram

5.4 Sequence Diagram

This traces the chronological "Ride Booking to Trip Start" workflow. It details the step-by-step exchange of HTTP REST requests and asynchronous Socket.io events between the clients, server, and database during a ride match.

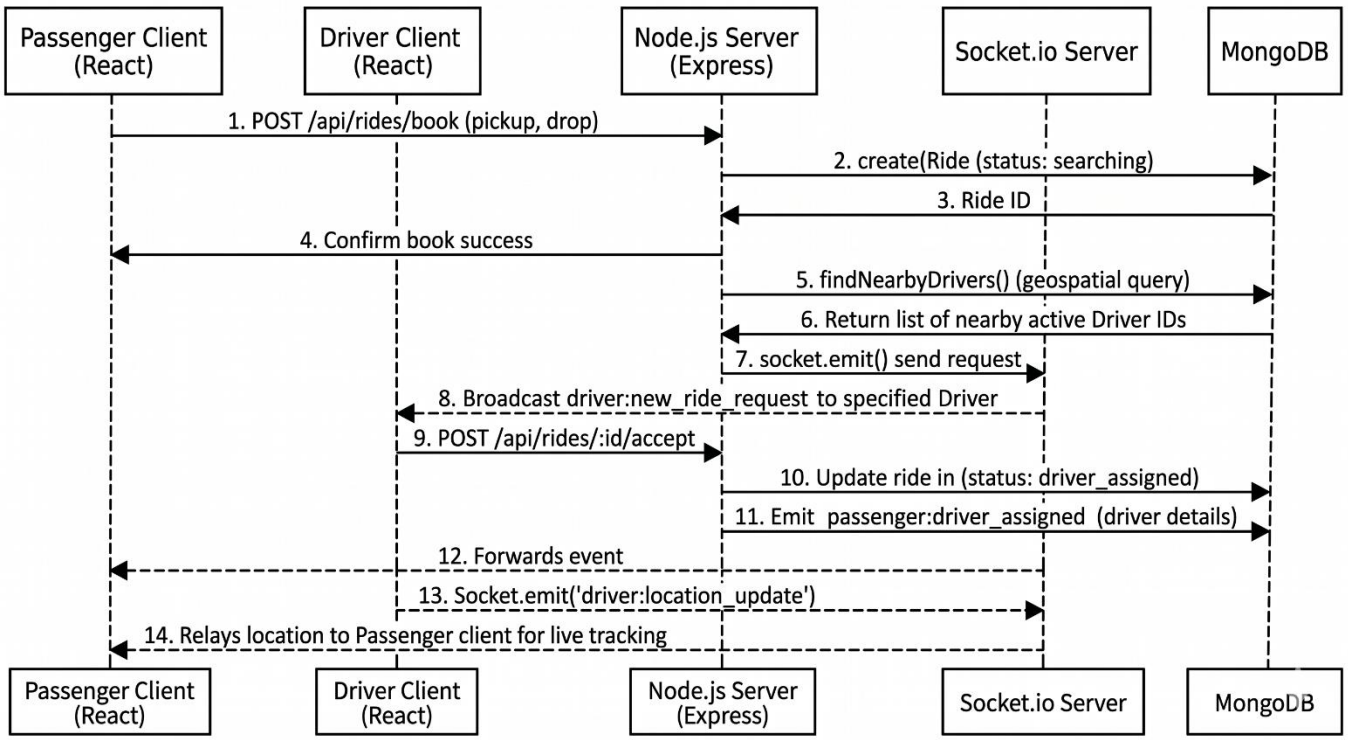

FIG 5.4 – Sequence Diagram

5.5 Component Diagram

The component diagram shows the physical software structure, detailing how React frontend modules (Auth, Dashboard) communicate with specific Node.js backend services (Pricing, Verification) and the MongoDB cluster.

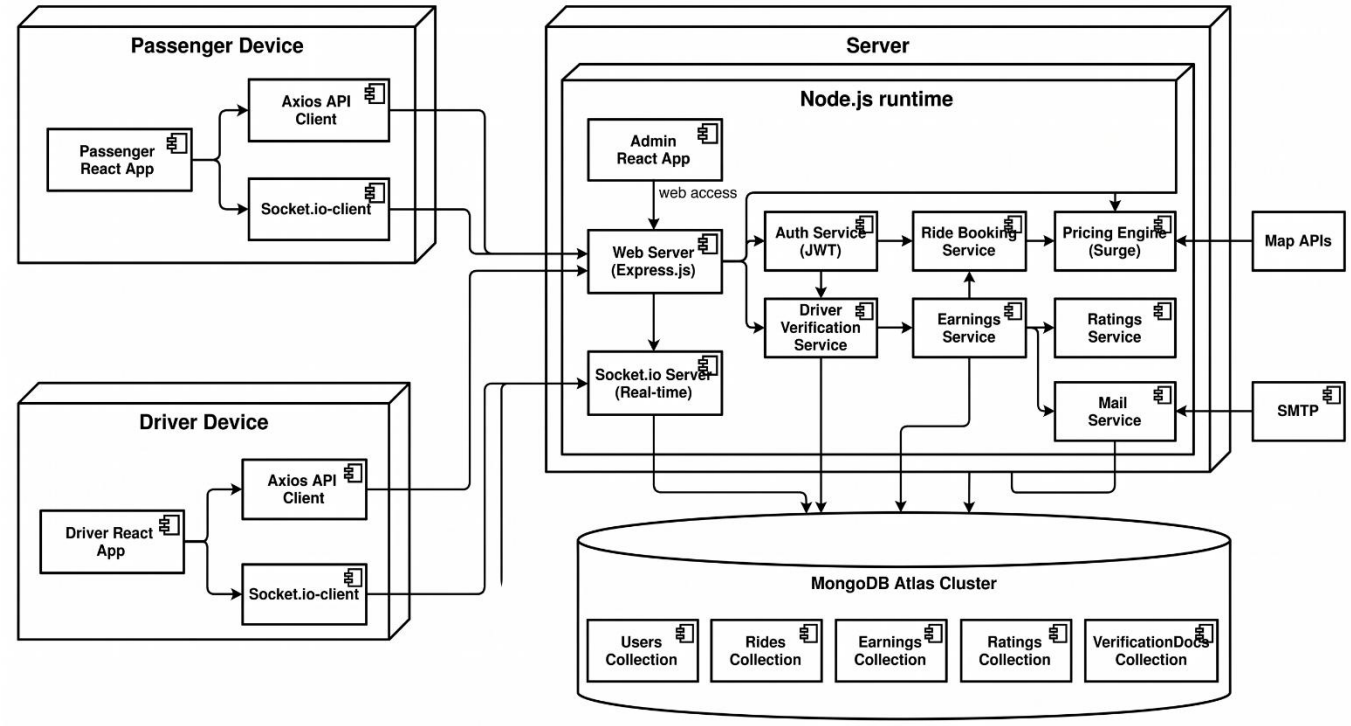

FIG 5.5 – Component Diagram

6. DATABASE DESIGN

6.1 ER Diagram

The Entity-Relationship Diagram provides a conceptual blueprint of the NoSQL database. It illustrates core entities like Users, Rides, Earnings, and Ratings, defining the cardinality of their relationships across the system.

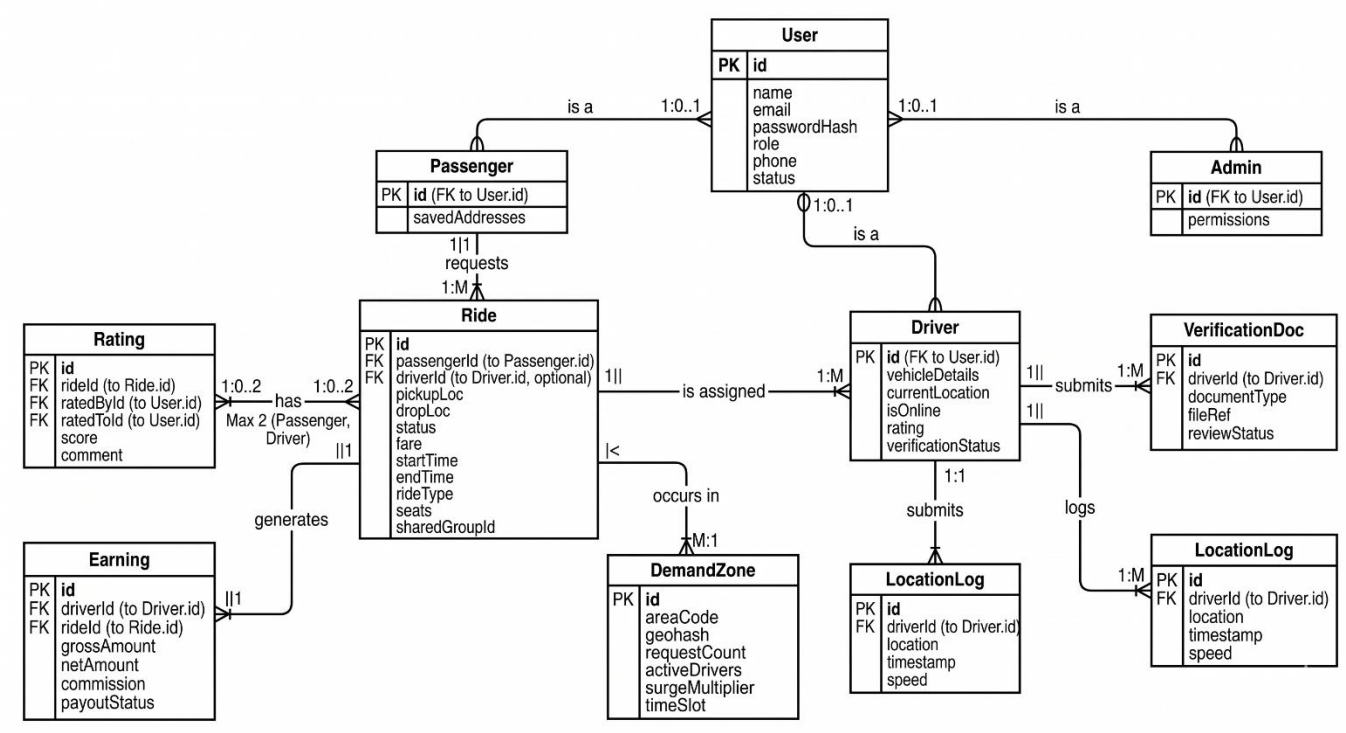

FIG 6.1 – Entity Relationship Diagram

6.2 Schema Diagram

This visualizes the technical BSON document structures for the MongoDB collections. It outlines the exact fields, data types (including GeoJSON Points for mapping), and embedded sub-documents used for storage.

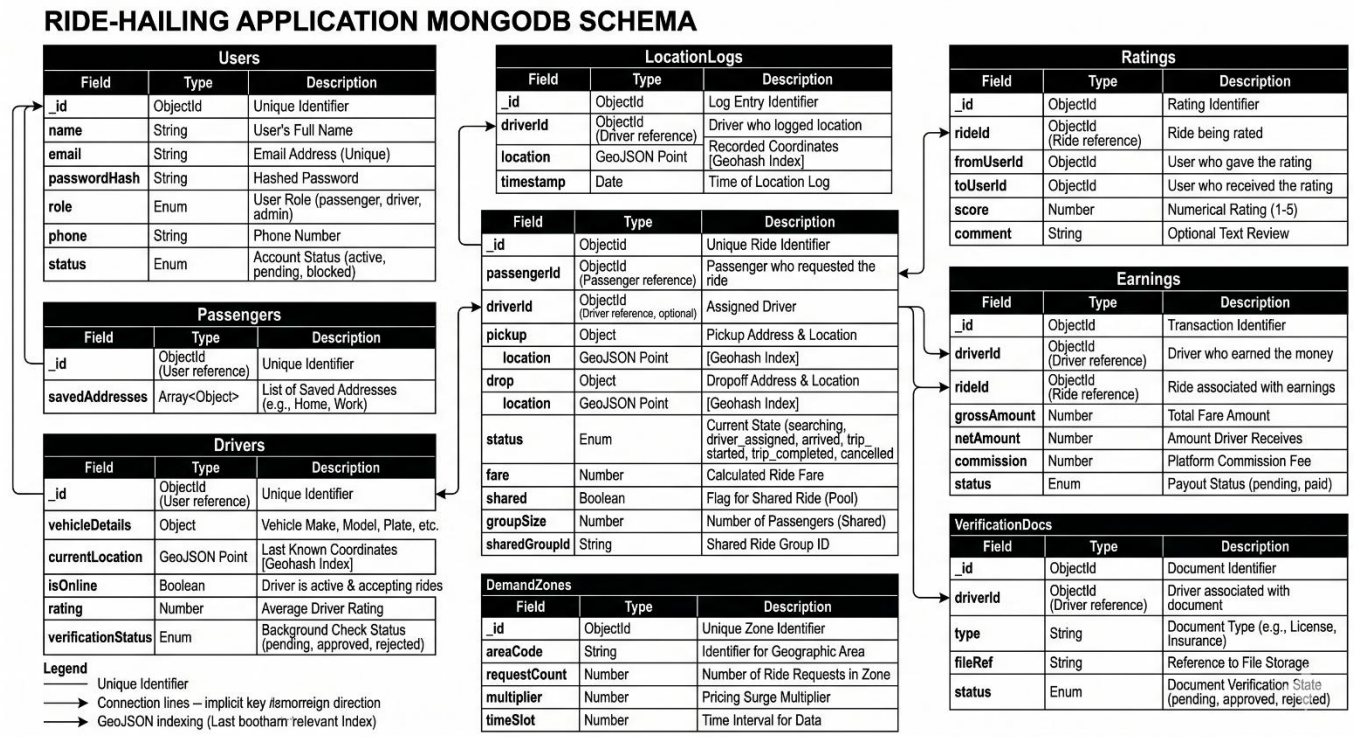

FIG 6.2 – Schema Diagram

6.3 Table Structures

This section lists the primary schema definitions (Users, Drivers, Rides, Earnings, VerificationDocs), detailing their specific attributes, primary/foreign keys (ObjectIds), and allowed Enum values.

Users Collection

_id	ObjectId	(Primary Key)
name	String	(User's full name)
email	String	(Unique email address)
passwordHash	String	(Hashed password)
role	Enum	('passenger', 'driver', 'admin')
phone	String	(Contact number)
status	Enum	('active', 'pending', 'blocked')

Passengers Collection

_id	ObjectId	(Primary Key, references Users)
savedAddresses	Array	(Favorite locations)

Drivers Collection

_id	ObjectId	(Primary Key, references Users)
vehicleDetails	Object	(Make, model, plate, etc.)
currentLocation	GeoJSON Point	(2dsphere indexed)
isOnline	Boolean	(Availability status)
verificationStatus	Enum	('pending', 'approved', 'rejected')

Rides Collection

_id	ObjectId	(Primary Key)
passengerId	ObjectId	(References Passengers)
driverId	ObjectId	(References Drivers, optional)
pickup	Object	(Address, GeoJSON Point)
drop	Object	(Address, GeoJSON Point)
status	Enum	(searching, assigned, started, completed, etc.)
fare	Number	(Calculated trip cost)

Drivers Collection

_id	ObjectId	(Primary Key, references Users)
savedAddresses	Array	(Favorite locations)

Rides Collection

_id	ObjectId	(Primary Key)
passengerId	ObjectId	(References Passengers)
driverId	ObjectId	(References Drivers, optional)
pickup	Object	(Address, GeoJSON Point)
drop	Object	(Address, GeoJSON Point)
status	Enum	(searching, assigned, started, completed, etc.)
fare	Number	(Calculated trip cost)

Ratings Collection

_id	ObjectId	(Primary Key)
rideId	ObjectId	(References Rides)
from	ObjectId	(User who rated)
to	ObjectId	(User who was rated)
score	Number	(1-5)

Earnings Collection

_id	ObjectId	(Primary Key)
driverId	ObjectId	(References Drivers)
rideId	ObjectId	(References Rides)
netAmount	Number	(Driver's share)

VerificationDocs Collection

_id	ObjectId	(Primary Key)
driverId	ObjectId	(References Drivers)
type	String	(e.g., license)
fileRef	String	(Path to file)

...Other collections similarly listed.

FIG 6.3 – Table Structure Diagram

6.4 Relationship Mapping

This mapping explains how isolated NoSQL documents are linked together using ObjectIds. For example, it shows how a specific Ride document is explicitly tied to its corresponding Earning and Rating records to form a fully traceable data network.

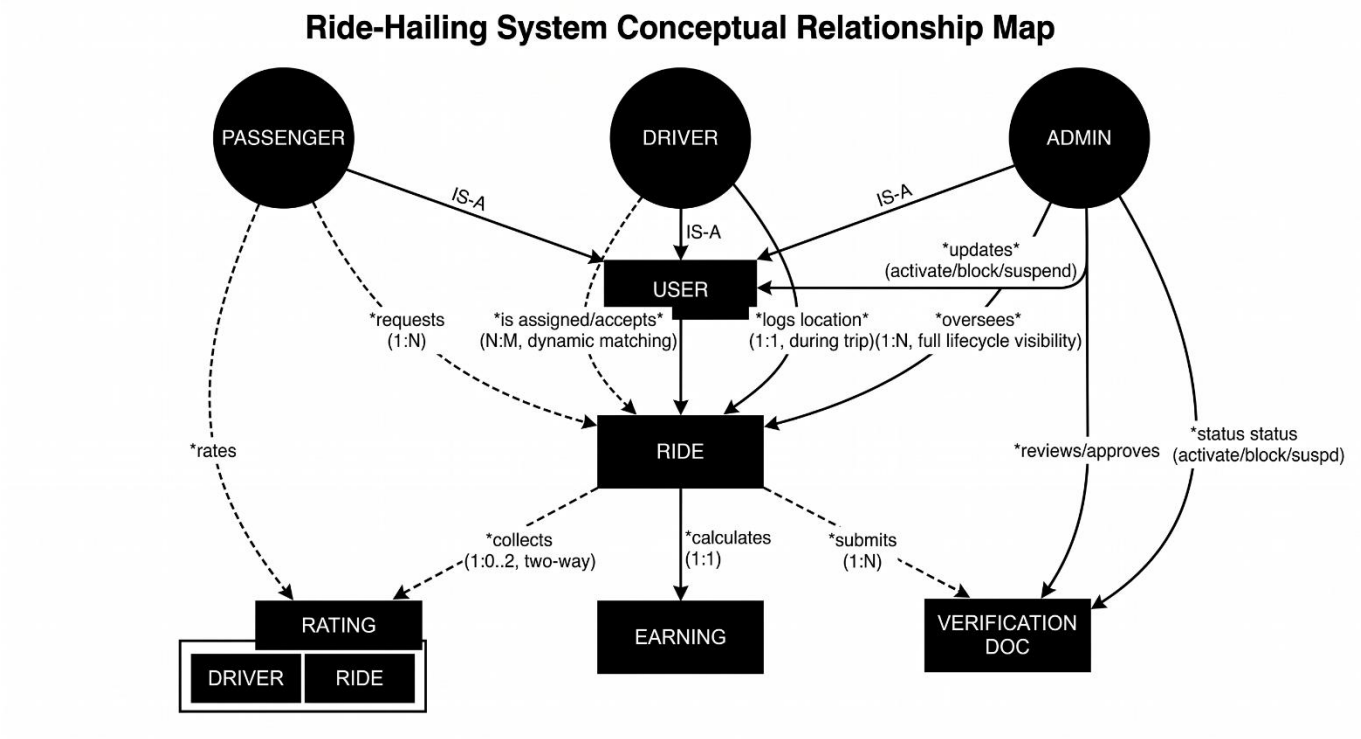

FIG 6.4 – Relationship Mapping Diagram